

## WM9PG-15 Cloud & Virtualisation Security Report

5750945

# Contents

|                                                                |    |
|----------------------------------------------------------------|----|
| Executive Summary .....                                        | 1  |
| 1 Section 1 – Test Case Definition and Outcomes.....           | 1  |
| 2 Section 2 – Image Hardening and Generation .....             | 6  |
| 2.1 Backend Hardening .....                                    | 6  |
| 2.1.1 TC-06 Pinning the Base Image .....                       | 6  |
| 2.1.2 TC -01 Removing Hardcoded Credentials .....              | 6  |
| 2.1.3 TC -05 Enforcing Non-Root Execution .....                | 8  |
| 2.1.4 Functional Verification .....                            | 10 |
| 2.2 Frontend Hardening .....                                   | 10 |
| 2.2.1 TC -04 Upgrading the End-of-Life Base Image .....        | 10 |
| 2.2.2 TC- 08 Restricting Files Copied into the Web Root .....  | 12 |
| 2.2.3 TC - 05 Enforcing Non-Root Execution .....               | 14 |
| 2.2.4 Functional Verification .....                            | 16 |
| 2.2.5 TC-10 HEALTHCHECK Instructions .....                     | 16 |
| 3 Section 3 – Runtime Hardening and Verification .....         | 20 |
| 3.1 TC-02 Separating Services into Individual Containers ..... | 20 |
| 3.2 TC-03 Removing Exposed Database Port .....                 | 22 |
| 3.3 TC-7 Network Isolation.....                                | 23 |
| 3.4 TC-09 Resource Limits.....                                 | 27 |
| 3.5 Final Testing .....                                        | 29 |
| Conclusion .....                                               | 30 |
| References .....                                               | 31 |
| Appendices .....                                               | 32 |

## Executive Summary

This report presents a security assessment of the organisation's containerised Todo list application, which consists of an Apache/PHP frontend, a Go API backend and a PostgreSQL database. The assessment identified ten security weaknesses in the original application. These included database credentials stored in a file, unnecessary exposure of the database port, the use of outdated software versions, a lack of network segmentation and other issues. These weaknesses could increase the risk of unauthorised access, data breaches or disrupt the application.

A structured methodology was followed, with ten test cases defined using guidance from NIST SP 800-190 and CIS Docker benchmark. These test cases were evaluated against the original deployment using a combination of automated tools and manual verification. Hardening measures were then implemented to address all identified weaknesses. These included securing database credentials, applying memory limits and restricting communication between application components.

After the security improvements were applied, testing confirmed that the application continued to operate as expected with no impact on user experience or functionality. These improvements strengthened the overall security posture of the system and reduced its exposure to common security risks.

## 1 Section 1 – Test Case Definition and Outcomes

The assessment was conducted using a five-phase methodology. First, the application was analysed to understand its components and architecture. Test cases were then created using guidance from NIST SP 800-190 and CIS Docker Benchmark. Evidence was gathered through a combination of manual inspection and automated tools like Trivy, Hadolint and Dockle. The identified issues were then addressed by applying security hardening measures to the Dockerfiles and configurations within the 'hardened' folder of the project. Finally, the application was re-tested to verify that all functionality remained intact after the security improvements were applied. This approach ensures that every finding is supported by evidence.

| No    | Test Case                                     | Description                                                                                                                                                                                                                                 | Expected Result                                       | Actual Result                                                                                                                                                                                                                                                       | Pass /Fail | Risk Rating |
|-------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-------------|
| TC-01 | Hardcoded database credentials in Docker File | Database credentials were hardcoded as variables in the docker file. The CIS Docker (Sec 4.10) (CIS, 2025) states that sensitive information must not be stored in Dockerfile, as anyone with access to the image can view the credentials. | Credentials should not be visible in the Docker file. | POSTGRES_PASSWORD=postgres and POSTGRES_USER=postgres were found hardcoded on of the backend Dockerfile and visible via docker inspect and cat.<br><br>Furthermore, the username and password were identical which increases the likelihood of brute-force attacks. | Fail       | Critical    |
| TC-02 | Multiple services in one container            | The backend container runs PostgreSQL and Go API within a single container, running multiple services. It is better to make separate containers to manage, update and troubleshoot services.                                                | Each container should run a single service.           | Docker top exercise-backend and start.sh showed both Postgres and Go API server running inside the same container.                                                                                                                                                  | Fail       | High        |
| TC-03 | Database port unnecessarily exposed to host   | The exposed port 5432 is mapped directly to the host machine, which allows direct access to the database. This does not comply with CIS Docker (sec 5.9) (CIS, 2025)                                                                        | Port 5432 should not be mapped to the host.           | Docker ps confirmed 0.0.0.0:5432->5432/t<br><br>A direct connection was also established using psql -h localhost -p 5432 -U postgres, which confirmed the database port is publicly accessible from the host.                                                       | Fail       | Critical    |
| TC-04 | End-of-life PHP used in frontend              | The fronted Dockerfile uses PHP 7.4 which has reached its end -of life in November 2022 and no longer receives security                                                                                                                     | The base image should be actively maintained          | Frontend Dockerfile uses 'FROM php:7.4' Apache which received end-of-life in November 2022.                                                                                                                                                                         | Fail       | High        |

|       |                                                 |                                                                                                                                                                                                                                                                                     |                                                                                          |                                                                                                                                                                                                                                         |      |          |
|-------|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|----------|
|       |                                                 | updates. (Karunaratne, 2022). This violates the NIST 8000-190 (Sec 3.11) (Souppaya, Morello and Scarfone, 2017)                                                                                                                                                                     | with current version.                                                                    | Trivy scan (Figure 54) identified 7,189 vulnerabilities including 54 critical and 1,440 high severity findings. While many of these relate Apache and Debian, the use of unsupported PHP adds vulnerabilities to the system.            |      |          |
| TC-05 | Containers running as root user                 | Neither container defines a non-root user, so all process run with full root privileges. If an attacker exploits a vulnerability and escapes the container, they will gain full access to the host.<br><br>The CIS sec 4.1 (CIS, 2025) says containers should run as non-root user. | Both containers should run as a non-root user.                                           | Docker exec exercise-backend whoami and docker exec exercise-frontend whoami both returned roots. Neither Dockerfile contains a USER instruction.<br><br>Dockle confirmed this with a warning CIS-DI-001 'Last user should not be root' | Fail | Critical |
| TC-06 | Latest image tag used in backend Dockerfile     | The backend Dockerfile uses 'From Postgres: latest' which does not specify a fixed PostgreSQL version. The 'latest' can introduce compatibility issues or new vulnerabilities.                                                                                                      | All base images should use a specific version.                                           | Backend docker file contains 'From Postgres: latest.'<br><br>Hadolint confirmed this with 'warning DL3007: using latest is prone to errors.'                                                                                            | Fail | Medium   |
| TC-07 | Absence of network isolation between containers | The containers share the default docker bridge network with no custom network segmentation. Without network isolation, an attacker who compromise the frontend container may be able to                                                                                             | Frontend and backend should be on separate networks. The frontend should not communicate | Both containers share the default Docker bridge network with no custom network configuration.<br><br>Docker inspect confirmed both containers were assigned to the same default bridge network.                                         | Fail | High     |

|       |                                                 |                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                         |                                                                                                                                                                                                                                                                                  |      |        |
|-------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|--------|
|       |                                                 | access the backend database container.                                                                                                                                                                                                                                                                                                                                            | to the database directly.                                                                               |                                                                                                                                                                                                                                                                                  |      |        |
| TC-08 | Overly permissive file permissions on web files | The frontend Dockerfile uses <code>chmod -R 775</code> on the web directory, which gives read and executable permissions to all users in the container. While this allows the web server to access the files, it also gives unnecessary permission to other users. These excessive permissions could allow an attacker who gains access to the container to read sensitive files. | Web files should be owned by <code>www-data</code> and have permissions less than 775.                  | Docker exec exercise-frontend <code>ls -la /var/www/html</code> showed all files assigned permissions of 755. The docker file was found inside <code>/var/www/html</code> .<br><br>Dockle flagged this under DKL-LI-0003 unnecessary file <code>var/www/html/Dockerfile</code> . | Fail | High   |
| TC-09 | No CPU or memory limits applied to containers   | No CPU limits have been configured for the containers. This means a container can use unlimited system resources, which can cause system instability. This violates The CIS docker sec 5.11 (CIS, 2025)                                                                                                                                                                           | Both containers should have memory and CPU limits set.                                                  | Docker inspect showed <code>Memory:0</code> and <code>NanoCPU:0</code> for both the containers confirming no resource limits is applied.                                                                                                                                         | Fail | High   |
| TC-10 | No HEALTHCHECK instructions in docker files     | The docker images does not include a HEALTHCHECK instructions. This means docker can only verify that the container is running and not if the application is working correctly.                                                                                                                                                                                                   | Both docker files should include a health check instruction that regularly test whether the application | Neither Dockerfile contains HEALTHCHECK instruction.<br><br>Dockle confirmed this for both images with check CIS-DI-006: not found HEALTHCHECK statement.                                                                                                                        | Fail | Medium |

|  |  |                                                                                                                                                                                       |                          |  |  |  |
|--|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|--|--|--|
|  |  | As a result, users could experience service disruptions without any alerts. This does not comply with The CIS Docker (Sec 4.6) (CIS, 2025) which recommends implementing HEALTHCHECKS | is responding correctly. |  |  |  |
|--|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|--|--|--|

## 2 Section 2 – Image Hardening and Generation

### 2.1 Backend Hardening

#### 2.1.1 TC-06 Pinning the Base Image

The original backend Dockerfile used Postgres: latest. The image was updated to postgres:18.4 to use a specific PostgreSQL version rather than the latest tag. This reduces the risk of unexpected behaviour from automatic updates as newer versions become available.

```
(root@kali)~[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# hadolint backend/Dockerfile
backend/Dockerfile:3 DL3008 warning: Pin versions in apt get install. Instead of 'apt-get install <package>' use 'apt-get install <package>=<version>'
backend/Dockerfile:3 DL3015 info: Avoid additional packages by specifying '--no-install-recommends'
backend/Dockerfile:13 DL3007 warning: Using latest is prone to errors if the image will ever update. Pin the version explicitly to a release tag
```

Figure 1 - Hadolint warning DL3007 - progress:latest tag

```
Session Actions Edit View Help
GNU nano 8.7.1 hardened/backend/Dockerfile
FROM golang:1.25-bookworm AS builder

RUN apt-get update && apt-get upgrade -y && apt-get install -y \
    git \
    build-essential && \
    rm -rf /var/lib/apt/lists/*
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags "-s -w" -o /server ./

FROM postgres:18.4
ARG POSTGRES_DB=exercise
ENV POSTGRES_DB=${POSTGRES_DB}

# non-root user for GO API server
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

COPY --from=builder /server /server
COPY start.sh /start.sh
RUN chmod +x /start.sh /server && \
    chown appuser:appgroup /server /start.sh

EXPOSE 8081
USER appuser
HEALTHCHECK --interval=50s --timeout=5s --retries=5 \
    CMD pg_isready -h "$DB_HOST" -p "$DB_PORT" || exit 1
ENTRYPOINT ["/start.sh"]
```

Figure 2 - postgres:18.4 relacing postgres:latest tag

#### 2.1.2 TC -01 Removing Hardcoded Credentials

TC-01 Identified that POSTGRES\_USER and POSTGRES\_PASSWORD were hardcoded in the Dockerfile which makes them visible within the build image and is recoverable via docker history.

```

(root@kali)~/Downloads/cvs_pma_2026/cvs_pma_2026/backend
# docker inspect exercise-backend | grep -A 10 "Env"
"Env": [
  "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/lib/postgresql/18/bin",
  "GOSU_VERSION=1.19",
  "LANG=en_US.utf8",
  "PG_MAJOR=18",
  "PG_VERSION=18.4-1.pgdg13+1",
  "PGDATA=/var/lib/postgresql/18/docker",
  "POSTGRES_USER=postgres",
  "POSTGRES_PASSWORD=postgres",
  "POSTGRES_DB=exercise"
],

```

Figure 3 - Harcoded credentials in Dockerfile

```

(root@kali)~/Downloads/cvs_pma_2026/cvs_pma_2026
# hadolint Dockerfile
Dockerfile
- CIS-D-0019: Do not store credential in environment variables/files
  * Suspicious ENV key found : key on RUN /bin/sh -c set -ex; key="B97B8AFC4A1A7F04F244A87FCC7D46ACCC4CFB"; export GNUPGHOME="$(mktemp -d)"; mkdir -p /usr/local/share/keyrings/; gpg --batch --keyserver keyserver.ubuntu.com --recv-keys "$key";
  * Suspicious ENV key found : POSTGRES_PASSWORD on ENV POSTGRES_PASSWORD=postgres (You can suppress it with --accept-key)
  * Suspicious ENV key found : POSTGRES_PASSWORD on ENV POSTGRES_PASSWORD=postgres (You can suppress it with --accept-key)

```

Figure 4 - Hadolint warning D-0010 for credentials stored in file

These were removed entirely from the Dockerfile and are now provided at runtime through an external .env file. This is verified using docker history which confirmed that the credentials do not appear in docker history.

```

FROM golang:1.15-bookworm AS builder
RUN apt-get update && apt-get install -y \
  git \
  build-essential && \
  rm -rf /var/lib/apt/lists/*
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags "-s -w" -o /server ./

FROM postgres:18.4
ARG POSTGRES_DB=exercise
ENV POSTGRES_DB=${POSTGRES_DB}

# non-root user for the API server
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

COPY --from=builder /server /server
COPY db-init.sql /docker-entrypoint-initdb.d/db-init.sql
COPY start.sh /start.sh
RUN chmod +x /start.sh /server && \
  chown appuser:appgroup /server

EXPOSE 8081 5432
ENTRYPOINT ["/start.sh"]

```

Figure 5 - No hardcoded credentials in DockerFile

```

GNU nano 8.7.1
DB_HOST=database
DB_USER=postgres
DB_PASSWORD=ap89lle673@
POSTGRES_USER=postgres
POSTGRES_PASSWORD=ap89lle673@
POSTGRES_DB=exercise
DB_PORT=5432
DB_NAME=exercise

```

Figure 6 - .env file

```

(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
# docker history exercise-backend:hardened
IMAGE          CREATED BY     SIZE    COMMENT
b4bd955c4e86   37 hours ago  ENTRYPOINT ["/start.sh"] 0B      buildkit.dockerfile.v0
<missing>      37 hours ago  HEALTHCHECK 6["CMD-SHELL" "pg_isready -h \_ 0B      buildkit.dockerfile.v0
<missing>      37 hours ago  USER appuser 0B      buildkit.dockerfile.v0
<missing>      37 hours ago  EXPOSE [8081/tcp] 0B      buildkit.dockerfile.v0
<missing>      37 hours ago  RUN [1 POSTGRES_DB=exercise /bin/sh -c chmod_ 10.3MB  buildkit.dockerfile.v0
<missing>      37 hours ago  COPY start.sh /start.sh # buildkit 294B    buildkit.dockerfile.v0
<missing>      3 days ago    COPY /server /server # buildkit 10.3MB  buildkit.dockerfile.v0
<missing>      3 days ago    RUN [1 POSTGRES_DB=exercise /bin/sh -c group_ 4.61kB  buildkit.dockerfile.v0
<missing>      3 days ago    ENV POSTGRES_DB=exercise 0B      buildkit.dockerfile.v0
<missing>      3 days ago    ARG POSTGRES_DB=exercise 0B      buildkit.dockerfile.v0
<missing>      10 days ago   CMD ["postgres"] 0B      buildkit.dockerfile.v0
<missing>      10 days ago   EXPOSE map[5432/tcp:{}] 0B      buildkit.dockerfile.v0
<missing>      10 days ago   STOPSIGNAL SIGINT 0B      buildkit.dockerfile.v0
<missing>      10 days ago   ENTRYPOINT ["docker-entrypoint.sh"] 0B      buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c ln -sT docker-ensure-initdb.s_ 23B     buildkit.dockerfile.v0
<missing>      10 days ago   COPY docker-entrypoint.sh docker-ensure-init_ 16.9kB  buildkit.dockerfile.v0
<missing>      10 days ago   VOLUME [/var/lib/postgresql] 0B      buildkit.dockerfile.v0
<missing>      10 days ago   ENV PGDATA=/var/lib/postgresql/18/docker 0B      buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c install --verbose --directory_ 0B      buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c set -eux; dpkg-divert --add _ 66.6kB  buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c set -eux; export PYTHONDONTW_ 329MB   buildkit.dockerfile.v0
<missing>      10 days ago   ENV PG_VERSION=18.4-1.pgdg13+1 0B      buildkit.dockerfile.v0
<missing>      10 days ago   ENV PATH=/usr/local/sbin:/usr/local/bin:/usr_ 0B      buildkit.dockerfile.v0
<missing>      10 days ago   ENV PG_MAJOR=18 0B      buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c set -eux; key='B97B0AFCAA1A47_ 3.98kB  buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c mkdir /docker-entrypoint-init_ 0B      buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c set -eux; apt-get update; a_ 3.38MB  buildkit.dockerfile.v0
<missing>      10 days ago   ENV LANG=en_US.utf8 0B      buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c set -eux; if [ -f /etc/dpkg/_ 25.6MB  buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c set -eux; savedAptMark="$(ap_ 3.63MB  buildkit.dockerfile.v0
<missing>      10 days ago   ENV GOSU_VERSION=1.19 0B      buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c set -eux; apt-get update; ap_ 15.9MB  buildkit.dockerfile.v0
<missing>      10 days ago   RUN /bin/sh -c set -eux; groupadd -r postgr_ 4.32kB  buildkit.dockerfile.v0
<missing>      11 days ago   # debian.sh --arch 'amd64' out/ 'trixie' '@l_ 78.6MB  debuerreotype 0.17

```

Figure 7 - docker history confirming credentials are not present

### 2.1.3 TC -05 Enforcing Non-Root Execution

TC-05 identified that the backend container was running as the root user.

```

(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
# docker exec exercise-backend whoami
root

```

Figure 8 - Backend root confirmation

While PostgreSQL already switches to the Postgres user, the custom Go API server continued to run with root privileges. To address this issue, a dedicated non-root user (appuser) was initially created, and the 'start.sh' was updated to run the server as 'appuser'. However, the container starts as root because it also launches PostgreSQL internally, which requires root privileges during startup. After PostgreSQL was moved into a separate container, the backend no longer needed root privileges. The start.sh was changed to wait for the database before launching Go server, and Dockerfile was updated with a user 'appuser'. This way, the container now runs as 'appuser' from the start rather than switching from root at runtime.

```

GNU nano 8.7.1 backend/Dockerfile
FROM golang:1.25-bookworm AS builder

RUN apt-get update && apt-get upgrade -y && apt-get install -y \
    git \
    build-essential && \
    rm -rf /var/lib/apt/lists/*
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags "-s -w" -o /server ./

FROM postgres:18.4
ARG POSTGRES_DB=exercise
ENV POSTGRES_DB=${POSTGRES_DB}

# non-root user for GO API server
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

COPY --from=builder /server /server
COPY start.sh /start.sh
RUN chmod +x /start.sh /server && \
    chown appuser:appgroup /server /start.sh

EXPOSE 8081
USER appuser
HEALTHCHECK --interval=50s --timeout=5s --retries=5 \
    CMD pg_isready -h localhost -p 5432 || exit 1
ENTRYPOINT ["/start.sh"]

```

Figure 9 - Hardened addition of non-root user

```

GNU nano 8.7.1 back
#!/bin/sh
set -e

# PostgreSQL now runs in a separate container ( see docker-compose.yml)

# Wait for PostgreSQL to become available before starting the Go backend
until pg_isready -h "$DB_HOST" -p "$DB_PORT" >/dev/null 2>&1; do
    echo "Waiting for database "
    sleep 1
done

exec /server

```

Figure 10 - Hardened start.sh

Verification was done using 'whoami' which returned appuser, while process confirmed that Go server and all other processes run under appuser account.

```

(root@kali)-[/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# docker exec exercise-backend-hardened whoami
appuser

(root@kali)-[/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# docker exec exercise-backend-hardened ps aux

```

| USER    | PID | %CPU | %MEM | VSZ     | RSS   | TTY | STAT | START | TIME | COMMAND |
|---------|-----|------|------|---------|-------|-----|------|-------|------|---------|
| appuser | 1   | 0.0  | 0.6  | 1235624 | 12180 | ?   | Ssl  | 20:35 | 0:00 | /server |
| appuser | 31  | 45.4 | 0.2  | 9404    | 4228  | ?   | Rs   | 20:37 | 0:00 | ps aux  |

Figure 11 - 'whoami' confirming non-root user

### 2.1.4 Functional Verification

To verify the hardening process did not affect the functionality, the backend API was tested using curl. The `/api/dbstatus` endpoint confirmed the database connectivity, and the `/api/todos` endpoint successfully retrieved the existing data.

```
(root@kali) ~ [~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# curl http://localhost:8081/api/dbstatus
{"db":"ok","row_count":1,"time":"2026-06-17T20:28:49Z"}

(root@kali) ~ [~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# curl http://localhost:8081/api/todos
[{"id":1,"title":"Welcome to your todo list","done":false,"created_at":"2026-06-17T20:28:27.395601Z"}]
```

Figure 12 – Curl verification of backend after changes

## 2.2 Frontend Hardening

### 2.2.1 TC -04 Upgrading the End-of-Life Base Image

TC-04 identified that the frontend dockerfile was using outdated `php:7.4-apache` image, which no longer receives security updates. The image was upgraded to `php:8.5-apache`.

```
(root@kali) ~ [~/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/frontend]
# cat Dockerfile
FROM php:7.4-apache

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        curl \
    && rm -rf /var/lib/apt/lists/*

COPY apache.conf /etc/apache2/sites-available/000-default.conf
COPY . /var/www/html/

RUN a2ensite 000-default.conf \
    && a2enmod rewrite

RUN chmod -R 755 /var/www/html \
    && chown -R www-data:www-data /var/www/html

EXPOSE 80

CMD ["apache2ctl", "-D", "FOREGROUND"]
```

Figure 13 - End-of-life php version in Dockerfile

```
root@kali: /home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
Session Actions Edit View Help
GNU nano 8.7.1 hardened/frontend/Dockerfile
FROM php:8.5-apache

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        curl \
    && rm -rf /var/lib/apt/lists/*

COPY apache.conf /etc/apache2/sites-available/000-default.conf
COPY index.html style.css /var/www/html/

RUN a2ensite 000-default.conf \
    && a2enmod rewrite

RUN chmod -R 544 /var/www/html/* \
    && find /var/www/html -type d -exec chmod 755 {} \; \
    && chown -R www-data:www-data /var/www/html

EXPOSE 80
USER www-data
CMD ["apache2ctl", "-D", "FOREGROUND"]
```

Figure 14 - Hardend - updated version of php

Testing confirmed that the container started successfully after the upgrade. During the upgrade, an outdated LoadModule referencing libphp7.so was removed from apache.conf as PHP 8.x images load the PHP module automatically.

```
(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
docker logs frontend-test
[Thu Jun 18 12:49:02.489621 2026] [so:warn] [pid 8:tid 8] AH01574: module access_compat_module is already loaded, skipping
[Thu Jun 18 12:49:02.491441 2026] [so:warn] [pid 8:tid 8] AH01574: module alias_module is already loaded, skipping
[Thu Jun 18 12:49:02.491458 2026] [so:warn] [pid 8:tid 8] AH01574: module auth_basic_module is already loaded, skipping
[Thu Jun 18 12:49:02.491460 2026] [so:warn] [pid 8:tid 8] AH01574: module authn_core_module is already loaded, skipping
[Thu Jun 18 12:49:02.491462 2026] [so:warn] [pid 8:tid 8] AH01574: module authn_file_module is already loaded, skipping
[Thu Jun 18 12:49:02.491464 2026] [so:warn] [pid 8:tid 8] AH01574: module authz_core_module is already loaded, skipping
[Thu Jun 18 12:49:02.491467 2026] [so:warn] [pid 8:tid 8] AH01574: module authz_host_module is already loaded, skipping
[Thu Jun 18 12:49:02.491469 2026] [so:warn] [pid 8:tid 8] AH01574: module authz_user_module is already loaded, skipping
[Thu Jun 18 12:49:02.491471 2026] [so:warn] [pid 8:tid 8] AH01574: module autoindex_module is already loaded, skipping
[Thu Jun 18 12:49:02.491473 2026] [so:warn] [pid 8:tid 8] AH01574: module deflate_module is already loaded, skipping
[Thu Jun 18 12:49:02.491474 2026] [so:warn] [pid 8:tid 8] AH01574: module dir_module is already loaded, skipping
[Thu Jun 18 12:49:02.491476 2026] [so:warn] [pid 8:tid 8] AH01574: module env_module is already loaded, skipping
[Thu Jun 18 12:49:02.491478 2026] [so:warn] [pid 8:tid 8] AH01574: module filter_module is already loaded, skipping
[Thu Jun 18 12:49:02.491933 2026] [so:warn] [pid 8:tid 8] AH01574: module mime_module is already loaded, skipping
[Thu Jun 18 12:49:02.491944 2026] [so:warn] [pid 8:tid 8] AH01574: module mpm_prefork_module is already loaded, skipping
[Thu Jun 18 12:49:02.491950 2026] [so:warn] [pid 8:tid 8] AH01574: module negotiation_module is already loaded, skipping
[Thu Jun 18 12:49:02.491952 2026] [so:warn] [pid 8:tid 8] AH01574: module reqtimeout_module is already loaded, skipping
[Thu Jun 18 12:49:02.491954 2026] [so:warn] [pid 8:tid 8] AH01574: module setenvif_module is already loaded, skipping
[Thu Jun 18 12:49:02.491956 2026] [so:warn] [pid 8:tid 8] AH01574: module status_module is already loaded, skipping
[Thu Jun 18 12:49:02.510456 2026] [so:warn] [pid 8:tid 8] AH01574: module access_compat_module is already loaded, skipping
[Thu Jun 18 12:49:02.510477 2026] [so:warn] [pid 8:tid 8] AH01574: module alias_module is already loaded, skipping
[Thu Jun 18 12:49:02.510479 2026] [so:warn] [pid 8:tid 8] AH01574: module auth_basic_module is already loaded, skipping
[Thu Jun 18 12:49:02.510481 2026] [so:warn] [pid 8:tid 8] AH01574: module authn_core_module is already loaded, skipping
[Thu Jun 18 12:49:02.510483 2026] [so:warn] [pid 8:tid 8] AH01574: module authn_file_module is already loaded, skipping
[Thu Jun 18 12:49:02.510485 2026] [so:warn] [pid 8:tid 8] AH01574: module authz_core_module is already loaded, skipping
[Thu Jun 18 12:49:02.510486 2026] [so:warn] [pid 8:tid 8] AH01574: module authz_host_module is already loaded, skipping
[Thu Jun 18 12:49:02.510488 2026] [so:warn] [pid 8:tid 8] AH01574: module authz_user_module is already loaded, skipping
[Thu Jun 18 12:49:02.510490 2026] [so:warn] [pid 8:tid 8] AH01574: module autoindex_module is already loaded, skipping
[Thu Jun 18 12:49:02.510492 2026] [so:warn] [pid 8:tid 8] AH01574: module deflate_module is already loaded, skipping
[Thu Jun 18 12:49:02.510494 2026] [so:warn] [pid 8:tid 8] AH01574: module dir_module is already loaded, skipping
[Thu Jun 18 12:49:02.510495 2026] [so:warn] [pid 8:tid 8] AH01574: module env_module is already loaded, skipping
[Thu Jun 18 12:49:02.510497 2026] [so:warn] [pid 8:tid 8] AH01574: module filter_module is already loaded, skipping
[Thu Jun 18 12:49:02.510586 2026] [so:warn] [pid 8:tid 8] AH01574: module mime_module is already loaded, skipping
[Thu Jun 18 12:49:02.510590 2026] [so:warn] [pid 8:tid 8] AH01574: module mpm_prefork_module is already loaded, skipping
[Thu Jun 18 12:49:02.510592 2026] [so:warn] [pid 8:tid 8] AH01574: module negotiation_module is already loaded, skipping
[Thu Jun 18 12:49:02.510595 2026] [so:warn] [pid 8:tid 8] AH01574: module reqtimeout_module is already loaded, skipping
[Thu Jun 18 12:49:02.510597 2026] [so:warn] [pid 8:tid 8] AH01574: module setenvif_module is already loaded, skipping
[Thu Jun 18 12:49:02.510599 2026] [so:warn] [pid 8:tid 8] AH01574: module status_module is already loaded, skipping
[Thu Jun 18 12:49:02.539958 2026] [mpm_prefork:notice] [pid 8:tid 8] AH00163: Apache/2.4.67 (Debian) PHP/8.5.7 configured -- resuming normal operations
[Thu Jun 18 12:49:02.540164 2026] [core:notice] [pid 8:tid 8] AH00094: Command line: '/usr/sbin/apache2 -D FOREGROUND'
```

Figure 15 – docker logs - successful loading of frontend container

```

GNU nano 8.7.1 frontend/apache.conf
ServerRoot "/etc/apache2"

LoadModule access_compat_module /usr/lib/apache2/modules/mod_access_compat.so
LoadModule alias_module /usr/lib/apache2/modules/mod_alias.so
LoadModule auth_basic_module /usr/lib/apache2/modules/mod_auth_basic.so
LoadModule authn_core_module /usr/lib/apache2/modules/mod_authn_core.so
LoadModule authn_file_module /usr/lib/apache2/modules/mod_authn_file.so
LoadModule authz_core_module /usr/lib/apache2/modules/mod_authz_core.so
LoadModule authz_host_module /usr/lib/apache2/modules/mod_authz_host.so
LoadModule authz_user_module /usr/lib/apache2/modules/mod_authz_user.so
LoadModule autoindex_module /usr/lib/apache2/modules/mod_autoindex.so
LoadModule deflate_module /usr/lib/apache2/modules/mod_deflate.so
LoadModule dir_module /usr/lib/apache2/modules/mod_dir.so
LoadModule env_module /usr/lib/apache2/modules/mod_env.so
LoadModule filter_module /usr/lib/apache2/modules/mod_filter.so
LoadModule headers_module /usr/lib/apache2/modules/mod_headers.so
LoadModule mime_module /usr/lib/apache2/modules/mod_mime.so
LoadModule mpm_prefork_module /usr/lib/apache2/modules/mod_mpm_prefork.so
LoadModule negotiation_module /usr/lib/apache2/modules/mod_negotiation.so
#LoadModule php7_module /usr/lib/apache2/modules/libphp7.so

LoadModule reqtimeout_module /usr/lib/apache2/modules/mod_reqtimeout.so
LoadModule setenvif_module /usr/lib/apache2/modules/mod_setenvif.so
LoadModule status_module /usr/lib/apache2/modules/mod_status.so

```

Figure 16 - Load php module removed since no longer required

## 2.2.2 TC- 08 Restricting Files Copied into the Web Root

TC-08 identified that the Dockerfile copied all project files into the web root using COPY./var/www/html. As a result, unnecessary files such as the Dockerfile were included in the container's web-accessible directory.

```

(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
$ docker exec exercise-frontend ls -la /var/www/html
total 32
drwxr-xr-x 1 www-data www-data 4096 Jun 12 08:16 .
drwxr-xr-x 1 root root 4096 Nov 15 2022 ..
-rwxr-xr-x 1 www-data www-data 419 May 28 15:24 Dockerfile
-rwxr-xr-x 1 www-data www-data 2247 May 28 15:42 apache.conf
-rwxr-xr-x 1 www-data www-data 4076 May 28 15:56 index.html
-rwxr-xr-x 1 www-data www-data 102 May 28 13:46 style.css

```

Figure 17 – Original frontend showing Dockerfile along with index.html and style.css

```

(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026/frontend]
$ cat Dockerfile
FROM php:7.4-apache

RUN apt-get update \
    && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY apache.conf /etc/apache2/sites-available/000-default.conf
COPY . /var/www/html/

RUN a2ensite 000-default.conf \
    && a2enmod rewrite

RUN chmod -R 755 /var/www/html \
    && chown -R www-data:www-data /var/www/html

EXPOSE 80

CMD ["apache2ctl", "-D", "FOREGROUND"]

```

Copied all project files into web root

Permissions set to 755

Figure 18 - Original Dockerfile

To address this, the COPY was changed to copy only the files required by the website, index.html and style.css. This prevents configuration files and other unnecessary files from being exposed by an attacker.

The file permissions were also changed from 755 to 544 for files which removes unnecessary write and execute access.

```
root@kali: /home/kali/Downloads/cvs_pma_2
Session Actions Edit View Help
GNU nano 8.7.1 frontend/D
FROM php:8.5-apache

RUN apt-get update \
    && apt-get upgrade -y \
    && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY apache.conf /etc/apache2/sites-available/000-default.conf
COPY index.html style.css /var/www/html/

RUN a2ensite 000-default.conf \
    && a2enmod rewrite

RUN chmod -R 544 /var/www/html/* \
    && find /var/www/html -type d -exec chmod 755 {} \; \
    && chown -R www-data:www-data /var/www/html

EXPOSE 80
USER www-data

HEALTHCHECK --interval=50s --timeout=5s --retries=5 \
    CMD curl -f http://localhost/ || exit 1

CMD ["apache2ctl", "-D", "FOREGROUND"]
```

Figure 19 - Hardened Dockerfile

```
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# docker exec frontend-test ls -la /var/www/html
total 24
drwxr-xr-x 1 www-data www-data 4096 Jun 18 12:48 .
drwxr-xr-x 1 root root 4096 Jun 11 00:23 ..
-r-xr--r-- 1 www-data www-data 4076 Jun 18 12:31 index.html
-r-xr--r-- 1 www-data www-data 102 Jun 18 12:31 style.css
```

Figure 20 - Hardened frontend web root showing index.html and style.css only present.

While reviewing the apache.conf during the TC-08 remediation, it was observed that configuration explicitly removed standard security headers (X-frame options, X-content type Options and Referrer Policy) using Header unset. These headers were also reinstated as a hardening measure since without them, the application is more vulnerable to clickjacking and MIME attacks (OWASP, 2024). This improves the overall security posture of the application.

```

GNU nano 8.7.1 frontend/apache.conf
ServerRoot "/etc/apache2"

LoadModule access_compat_module /usr/lib/apache2/modules/mod_access_compat.so
LoadModule alias_module /usr/lib/apache2/modules/mod_alias.so
LoadModule auth_basic_module /usr/lib/apache2/modules/mod_auth_basic.so
LoadModule authn_core_module /usr/lib/apache2/modules/mod_authn_core.so
LoadModule authn_file_module /usr/lib/apache2/modules/mod_authn_file.so
LoadModule authz_core_module /usr/lib/apache2/modules/mod_authz_core.so
LoadModule authz_host_module /usr/lib/apache2/modules/mod_authz_host.so
LoadModule authz_user_module /usr/lib/apache2/modules/mod_authz_user.so
LoadModule autoindex_module /usr/lib/apache2/modules/mod_autoindex.so
LoadModule deflate_module /usr/lib/apache2/modules/mod_deflate.so
LoadModule dir_module /usr/lib/apache2/modules/mod_dir.so
LoadModule env_module /usr/lib/apache2/modules/mod_env.so
LoadModule filter_module /usr/lib/apache2/modules/mod_filter.so
LoadModule headers_module /usr/lib/apache2/modules/mod_headers.so
LoadModule mime_module /usr/lib/apache2/modules/mod_mime.so
LoadModule mpm_prefork_module /usr/lib/apache2/modules/mod_mpm_prefork.so
LoadModule negotiation_module /usr/lib/apache2/modules/mod_negotiation.so

#LoadModule php7_module /usr/lib/apache2/modules/libphp7.so

LoadModule reqtimeout_module /usr/lib/apache2/modules/mod_reqtimeout.so
LoadModule setenvif_module /usr/lib/apache2/modules/mod_setenvif.so
LoadModule status_module /usr/lib/apache2/modules/mod_status.so

User www-data
Group www-data

ServerAdmin admin@localhost
ServerName localhost

ServerTokens Prod
ServerSignature Off
TraceEnable Off

ErrorLog /proc/self/fd/2
TransferLog /proc/self/fd/1
LogLevel warn

<VirtualHost *:80>
    ServerAdmin admin@localhost
    DocumentRoot "/var/www/html"
    ServerName localhost

    <Directory "/var/www/html">
        Options Indexes FollowSymLinks Includes ExecCGI
        AllowOverride All
        Require all granted
    </Directory>

    DirectoryIndex index.html

    Header unset X-XSS-Protection
    Header unset Content-Security-Policy

    Header always set X-Frame-Options "SAMEORIGIN"
    Header always set X-Content-Type-Options "nosniff"
    Header always set Referrer-Policy "strict-origin-when-cross-origin"

    <IfModule autoindex_module>
        IndexOptions FancyIndexing VersionSort
    </IfModule>
</VirtualHost>

```

Figure 21 - Security headers set

```

(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# docker exec exercise-frontend-hardened cat /var/www/html/Dockerfile
cat: /var/www/html/Dockerfile: No such file or directory

(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/Dockerfile
404

```

Figure 22 - Verification confirming Dockerfile is not accessible

### 2.2.3 TC - 05 Enforcing Non-Root Execution

TC-05 identified the frontend container was running as root. This was confirmed using docker exec exercise-front end whoami and by a dockle warning.

```

(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# docker exec exercise-frontend whoami
root

```

Figure 23 - Frontend root confirmation

```

(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
dockle --input /tmp/backend.tar
2026-06-16T07:06:37.528-0400 FATAL unable to initialize a image struct: failed to initialize source: opening file "/tmp/backend.tar": open /tmp/backend.tar: no such file or directory

(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
dockle --input /tmp/frontend.tar
WARN - CIS-DI-0001: Create a user for the container
* Last user should not be root

INFO - CIS-DI-0003: Enable Content Trust for Docker
* export DOCKER_CONTENT_TRUST=1 before docker pull/build
INFO - CIS-DI-0005: Add HEALTHCHECK instruction to the container image
* not found HEALTHCHECK statement
INFO - CIS-DI-0008: Confirm safety of setuid/setgid files
* setuid file: urwxr-xr-x usr/bin/newgrp
* setgid file: grwxr-xr-x usr/bin/chage
* setuid file: urwxr-xr-x usr/bin/chsh
* setuid file: urwxr-xr-x usr/bin/passwd
* setuid file: urwxr-xr-x bin/mount
* setgid file: grwxr-xr-x sbin/unix_chkpwd
* setuid file: urwxr-xr-x usr/bin/gpasswd
* setgid file: grwxr-xr-x usr/bin/wall
* setuid file: urwxr-xr-x usr/bin/chfn
* setgid file: grwxr-xr-x usr/bin/expiry
* setuid file: urwxr-xr-x bin/su
* setuid file: urwxr-xr-x bin/umount
INFO - DKL-LI-0003: Only put necessary files
* unnecessary file : var/www/html/dockerfile

```

Figure 24 - Dockle warning DI-0001 for root user

To address this, the Dockerfile was updated to use the www-data user instead. It was later verified that the container and all Apache process now run under the www-data account. This follows the principle of least privilege and improves the overall container security.

```

GNU nano 8.7.1 hardened/frontend/
FROM php:8.5-apache

RUN apt-get update \
    && apt-get upgrade -y \
    && apt-get install -y --no-install-recommends \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY apache.conf /etc/apache2/sites-available/000-default.conf
COPY index.html style.css /var/www/html/

RUN a2ensite 000-default.conf \
    && a2enmod rewrite

RUN chmod -R 544 /var/www/html/* \
    && find /var/www/html -type d -exec chmod 755 {} \; \
    && chown -R www-data:www-data /var/www/html

EXPOSE 80
USER www-data

HEALTHCHECK --interval=50s --timeout=5s --retries=5 \
    CMD curl -f http://localhost/ || exit 1

CMD ["apache2ctl", "-D", "FOREGROUND"]

```

Figure 25 - Hardened - addition of non-root user

```

(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
# docker exec frontend-test whoami
www-data

(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
# docker exec frontend-test ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
www-data        1  0.0  0.0   2680  1860 ?        Ss   12:49    0:00 /bin/sh /usr/sbin/apache2ctl -D FOREGROUND
www-data        8  0.0  1.5 194020 32220 ?        S    12:49    0:00 /usr/sbin/apache2 -D FOREGROUND
www-data        9  0.0  0.6 194108 12676 ?        S    12:49    0:00 /usr/sbin/apache2 -D FOREGROUND
www-data       10  0.0  0.5 194060 11336 ?        S    12:49    0:00 /usr/sbin/apache2 -D FOREGROUND
www-data       11  0.0  0.5 194060 11336 ?        S    12:49    0:00 /usr/sbin/apache2 -D FOREGROUND
www-data       12  0.0  0.5 194060 11336 ?        S    12:49    0:00 /usr/sbin/apache2 -D FOREGROUND
www-data       13  0.0  0.5 194060 11336 ?        S    12:49    0:00 /usr/sbin/apache2 -D FOREGROUND
www-data       20 75.0  0.1   6400  3860 ?        Rs   12:50    0:00 ps aux

```

Figure 26 - Verification of www-data

## 2.2.4 Functional Verification

After the hardening process, the frontend was tested using curl. The HTML response confirmed that the Todo List APP loaded correctly, including the page title, input form and JavaScript code used to communicate with the backend API. This showed that no functionality was affected due to security changes.

```
(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
# curl http://localhost:8080
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Exercise Frontend</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Todo List App</h1>
  <div id="out">Loading ... </div>
  <div id="db">Checking DB ... </div>
  <form id="todoForm">
    <input id="todoTitle" type="text" placeholder="Add a new task" autocomplete="off" />
    <button type="submit">Add Todo</button>
  </form>
  <ul id="todoList"></ul>
  <script>
    const apiHost = 'http://localhost:8081/api';

    async function load() {
      document.getElementById('out').textContent = 'Loading todos ...';
      await refreshDb();
      await loadTodos();
    }

    async function fetchJson(url, options) {
      const res = await fetch(url, options);
      if (!res.ok) {
        const err = await res.text();
        throw new Error(err || res.statusText);
      }
      return res.status === 204 ? null : res.json();
    }

    async function refreshDb() {
      try {
        const j = await fetchJson(`${apiHost}/dbstatus`);
        document.getElementById('db').textContent = `DB: ${j.db} (${j.row_count} todos)`;
      } catch (e) {
        document.getElementById('db').textContent = 'DB unreachable';
      }
    }

    async function loadTodos() {
      try {
        const todos = await fetchJson(`${apiHost}/todos`);
        renderTodos(todos);
        document.getElementById('out').textContent = 'Todo list loaded';
      } catch (e) {
        document.getElementById('out').textContent = 'Could not load todos';
      }
    }

    function renderTodos(todos) {
      const list = document.getElementById('todoList');
      list.innerHTML = '';
      if (!todos || todos.length === 0) {
        list.innerHTML = '<li>No todos yet.</li>';
      }
      return;
    }
  </script>
</body>
</html>
```

Figure 27 - Functional Verification of frontend

## 2.2.5 TC-10 HEALTHCHECK Instructions

TC-10 was addressed by adding HEALTHCHECK instructions to both containers.

```
(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
# dockle --input /tmp/backend.tar
2026-06-16T07:06:37.528-0400  FATAL  unable to initialize a image struct: failed to initialize source: opening file "/tmp/backend.tar": open /tmp/backend.tar: no such file or directory

(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
# dockle --input /tmp/frontend.tar
WARN - CIS-DI-0001: Create a user for the container
* Last user should not be root
INFO - CIS-DI-0002: Enable Content trust for Docker
* export DOCKER_CONTENT_TRUST=1 before docker pull/build
INFO - CIS-DI-0006: Add HEALTHCHECK instruction to the container image
* not found HEALTHCHECK statement
INFO - CIS-DI-0008: Confirm safety of setuid/setgid files
```

Figure 28 -Frontened Dockle warning DI-0006 - absence of HEALTHCHECK instructions

```

root@kali:~/Downloads/cvs_pma_2026/cvs_pma_2026# dockle --input /tmp/backend.tar
WARN - CIS-DI-0010: Do not store credential in environment variables/files
* Suspicious ENV key found : key on RUN /bin/sh -c set -ex; keys='B97B8AFCAA1AA7F846F244A07FCC7D46ACCC4CFB'; export GNUPGHOME="$(mktemp -d)"; mkdir -p
> /usr/local/share/keyrings/postgres.pg.asc; gpgconf --kill all; rm -rf "$GNUPGHOME" # buildkit (You can suppress it with --accept-key)
* Suspicious ENV key found : POSTGRES_PASSWORD on ENV POSTGRES_PASSWORD=postgres (You can suppress it with --accept-key)
WARN - CIS-DI-0001: Create a user for the container
* Last user should not be root
INFO - CIS-DI-0005: Enable Content trust for Docker
* export DOCKER_CONTENT_TRUST=1 before docker pull/build
INFO - CIS-DI-0006: Add HEALTHCHECK instruction to the container image
* not found HEALTHCHECK statement
INFO - CIS-DI-0008: Confirm safety of setuid/setgid files

```

Figure 29 - -Backened Dockle warning DI-0006 - absence of HEALTHCHECK instructions

The backend health check uses pg\_isready to confirm that PostgreSQL is accepting connections (PostgreSQL Documentation, 2026) while frontend health check uses curl to verify the webpage is being served correctly. The checks run every 50 seconds and allow up to 5 retries before marking the container as unhealthy. The Docker inspect verified both containers as healthy. This allows Docker to detect application failures automatically instead of only checking whether the container process is running.

```

Session Actions Edit View Help
GNU nano 8.7.1
FROM golang:1.25-bookworm AS builder

RUN apt-get update && apt-get upgrade -y && apt-get install -y \
    git \
    build-essential && \
    rm -rf /var/lib/apt/lists/*
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -ldflags "-s -w"

FROM postgres:18.4
ARG POSTGRES_DB=exercise
ENV POSTGRES_DB=${POSTGRES_DB}

# non-root user for GO API server
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

COPY --from=builder /server /server
COPY start.sh /start.sh
RUN chmod +x /start.sh /server && \
    chown appuser:appgroup /server /start.sh

EXPOSE 8081
USER appuser
HEALTHCHECK --interval=50s --timeout=5s --retries=5 \
    CMD pg_isready -h "$DB_HOST" -p "$DB_PORT" || exit 1
ENTRYPOINT ["/start.sh"]

```

Figure 30 – Hardened - Backend HEALTHCHECK instructions



```
Frontend-test
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# docker inspect --format='{{json .State.Health}}' backend-test | jq
{
  "Status": "healthy",
  "FailingStreak": 0,
  "Log": [
    {
      "Start": "2026-06-18T15:53:20.144436501-04:00",
      "End": "2026-06-18T15:53:20.242985135-04:00",
      "ExitCode": 0,
      "Output": "localhost:5432 - accepting connections\n"
    }
  ]
}
```

Figure 33 - backend HEALTHY status

### 3 Section 3 – Runtime Hardening and Verification

The runtime configuration was implemented using a Docker compose file. The docker-compose.yml addresses the remaining test cases.

#### 3.1 TC-02 Separating Services into Individual Containers

TC-02 identified PostgreSQL and Go API server were running within the same container. This increases complexity and does not follow docker recommendation to separate services into individual containers (Docker Documentation, 2025).

```
(root@kali) ~/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
docker top exercise-backend
UID          PID          PPID         C           STIME        TTY          TIME         CMD
root         5632         5608         0           18:18        ?           00:00:00     /server
root         5708         5632         0           18:18        ?           00:00:00     postgres
999          5901         5708         0           18:18        ?           00:00:00     postgres: io worker 0
999          5902         5708         0           18:18        ?           00:00:00     postgres: io worker 1
999          5903         5708         0           18:18        ?           00:00:00     postgres: io worker 2
999          5904         5708         0           18:18        ?           00:00:00     postgres: checkpointer
999          5905         5708         0           18:18        ?           00:00:00     postgres: background writer
999          5907         5708         0           18:18        ?           00:00:00     postgres: walwriter
999          5908         5708         0           18:18        ?           00:00:00     postgres: autovacuum launcher
999          5909         5708         0           18:18        ?           00:00:00     postgres: logical replication launcher
999          5923         5708         0           18:18        ?           00:00:00     postgres: postgres exercise 127.0.0.1(51910) idle
```

Figure 34 - All running within the same container

PostgreSQL was moved into a dedicated database container, separating it from the backend API container. The backend was configured to connect to the database using the 'DB\_HOST=database' environment variable supplied through the .env file. A health check was also added to the database service, and the backend was configured to start only after the database reported a healthy status.

```

root@kali: /home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
Session Actions Edit View Help
GNU nano 8.7.1 docker-compose.yml
services:
  database:
    image: postgres:18.4
    container_name: exercise-database
    env_file: .env
    networks:
      - backend-network
    mem_limit: 256m
    cpus: 0.5
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 50s
      timeout: 5s
      retries: 3
    volumes:
      - postgres-data:/var/lib/postgresql
      - ./backend/db-init.sql:/docker-entrypoint-initdb.d/01-db-init.sql:ro

  backend:
    image: exercise-backend:hardened
    container_name: exercise-backend-hardened
    env_file: .env
    ports:
      - "8081:8081"
    networks:
      - backend-network
      - frontend-network
    mem_limit: 512m
    cpus: 0.5
    depends_on:
      database:
        condition: service_healthy
    restart: unless-stopped

  frontend:
    image: exercise-frontend:hardened
    container_name: exercise-frontend-hardened
    ports:
      - "8080:80"
    networks:
      - frontend-network
    mem_limit: 128m
    cpus: 0.25
    depends_on:
      - backend
    restart: unless-stopped

networks:
  backend-network:
    driver: bridge
  frontend-network:
    driver: bridge

volumes:
  postgres-data:

```

Figure 35 - docker-compose.yml

Docker compose ps confirmed the application was running across three separate containers.

```

(root@kali) - [~/Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
$ docker compose ps

```

| NAME                       | IMAGE                      | COMMAND                  | SERVICE  | CREATED       | STATUS                      | PORTS                  |
|----------------------------|----------------------------|--------------------------|----------|---------------|-----------------------------|------------------------|
| exercise-backend-hardened  | exercise-backend:hardened  | "/start.sh"              | backend  | 2 minutes ago | Up About a minute (healthy) | 5432/tcp, 0.0.0.0:8081 |
| exercise-database          | postgres:18.4              | "docker-entrypoint.s..." | database | 2 minutes ago | Up 2 minutes (healthy)      | 5432/tcp               |
| exercise-frontend-hardened | exercise-frontend:hardened | "docker-php-entrypoi..." | frontend | 2 minutes ago | Up About a minute (healthy) | 0.0.0.0:8080->80/tcp,  |

Figure 36 - Separate containers with healthy status

Functional testing showed the new Todo entries could be created and retrieved successfully, which confirmed the backend communicated correctly with the database container.

```

(root@kali) - [~/Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
$ curl -X POST http://localhost:8081/api/todos -H "Content-Type: application/json" -d '{"title": "TC-02 fixed - services now separated!"}'
{"id":1,"title":"TC-02 fixed - services now separated!","done":false,"created_at":"2026-06-18T21:52:03.859303Z"}

```

Figure 37 - Testing todo entries

```
(root@kali)-[/home/./Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# curl http://localhost:8081/api/todos
[{"id":1,"title":"TC-02 fixed - services now separated!","done":false,"created_at":"2026-06-18T21:52:03.859303Z"}]
```

Figure 38 -Displaying todo entries

### 3.2 TC-03 Removing Exposed Database Port

TC-03 identified that PostgreSQL database was exposed to the host through port mapping 5432:5432, allowing direct access from outside the container environment.

```
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# cat Makefile
IMAGE_BACKEND = exercise-backend:latest
IMAGE_FRONTEND = exercise-frontend:latest

.PHONY: build build-backend build-frontend run run-backend run-frontend stop clean
build: build-backend build-frontend
build-backend:
    docker build -t $(IMAGE_BACKEND) ./backend
build-frontend:
    docker build -t $(IMAGE_FRONTEND) ./frontend
run: run-backend run-frontend
run-backend:
    docker run -d --rm --name exercise-backend \
        -e POSTGRES_USER=postgres \
        -e POSTGRES_PASSWORD=postgres \
        -e POSTGRES_DB=exercise \
        -p 8081:8081 \
        -p 5432:5432 $(IMAGE_BACKEND)
run-frontend:
    docker run -d --rm --name exercise-frontend -p 8080:80 $(IMAGE_FRONTEND)
stop:
    docker stop exercise-backend exercise-frontend || true
clean: stop
    docker rmi -f $(IMAGE_BACKEND) $(IMAGE_FRONTEND) || true
```

Figure 39 - Original database port exposed

```
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# psql -h localhost -p 5432 -U postgres -d exercise

Password for user postgres:
psql (18.3 (Debian 18.3-1+b1), server 18.4 (Debian 18.4-1.pgdg13+1))
Type "help" for help.

exercise=# \dt
          List of tables
 Schema | Name  | Type  | Owner
-----+-----+-----+-----
 public | todos | table | postgres
(1 row)

exercise=# SELECT * FROM todos;
 id | title | done | created_at
----+-----+-----+-----
  1 | Welcome to your todo list | f | 2026-06-15 22:18:17.613605+00
(1 row)

exercise=# \q

(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
#
```

Figure 40 - Connection to psql

To address this issue, the port mapping was removed from the hardened container on the internal application network.

The curl <http://localhost:5432> confirmed that the database was no longer reachable from the host, while the backend continued to communicate successfully with the database through the internal network. This reduced the application's external attack surface without affecting the functionality.

```
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# curl http://localhost:5432
curl: (7) Failed to connect to localhost port 5432 after 1 ms: Could not connect to server
```

Figure 41 - Verification of database connection failure

### 3.3 TC-7 Network Isolation

TC-07 identified that the containers were using Docker's default bridge network which allowed unrestricted communication between containers.

```
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# docker network ls
NETWORK_ID      NAME      DRIVER      SCOPE
9c76dcff7f08    bridge    bridge      local
211664f2c993    host      host        local
640da9e6ac25    none      null        local
```

Figure 42 - Original - lack of network isolation

```
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# docker inspect exercise-frontend | grep -A 20 Networks
"Networks": {
  "bridge": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": null,
    "MacAddress": "36:9d:3f:2f:20:3f",
    "DriverOpts": null,
    "GwPriority": 0,
    "NetworkID": "9c76dcff7f08db7f72d76365c79616c947d1b6e16c990802150f55d08d73dff1",
    "EndpointID": "d43df4cd1d92dd12c213c9984946da6b6b9d7b35dca820d739eefc08eacaa3be",
    "Gateway": "172.17.0.1",
    "IPAddress": "172.17.0.3",
    "IPPrefixLen": 16,
    "IPv6Gateway": "",
    "GlobalIPv6Address": "",
    "GlobalIPv6PrefixLen": 0,
    "DNSNames": null
  }
}
```

Figure 43 - Original - default bridge network

To address this issue, two separate Docker networks were defined – frontend-network and backend-network in the docker-compose.yml. The frontend container was assigned only to frontend-network, while the database container was assigned only to the backend-network. The backend service was connected to both networks since it requires requests from the front-end and send queries to the database. The change was made so that if one network is compromised, it does not automatically grant access to another. This approach aligns with NIST SP 800-190 (Section 4.4.2) (Souppaya, Morello and Scarfone, 2017).

```
root@kali: /home/kali/Downloads/cvs_pma_2026/cvs_pma_2
Session Actions Edit View Help
GNU nano 8.7.1 docker-compose.yml
services:
  database:
    image: postgres:18.4
    container_name: exercise-database
    env_file: .env
    networks:
      - backend-network
    mem_limit: 256m
    cpus: 0.5
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 50s
      timeout: 5s
      retries: 5
    volumes:
      - postgres-data:/var/lib/postgresql
      - ./backend/db-init.sql:/docker-entrypoint-initdb.d/01-db-init.sql:ro

  backend:
    image: exercise-backend:hardened
    container_name: exercise-backend-hardened
    env_file: .env
    ports:
      - "8081:8081"
    networks:
      - backend-network
      - frontend-network
    mem_limit: 512m
    cpus: 0.5
    depends_on:
      database:
        condition: service_healthy
    restart: unless-stopped

  frontend:
    image: exercise-frontend:hardened
    container_name: exercise-frontend-hardened
    ports:
      - "8080:80"
    networks:
      - frontend-network
    mem_limit: 128m
    cpus: 0.25
    depends_on:
      - backend
    restart: unless-stopped

networks:
  backend-network:
    driver: bridge
  frontend-network:
    driver: bridge

volumes:
  postgres-data:
```

Figure 44 - docker-compose.yml

This was verified using *docker inspect*, which confirmed that the frontend and database container do not communicate directly.

```
(root@kali)-[/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# docker inspect exercise-frontend-hardened | grep -A 50 "Networks"
"Networks": {
  "hardened_frontend-network": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": [
      "exercise-frontend-hardened",
      "frontend"
    ],
    "MacAddress": "12:0c:3d:bc:4d:0a",
    "DriverOpts": null,
    "GwPriority": 0,
    "NetworkID": "138b3f3ebd96c910f86ee83247659d71aa1995447a35f0016106e10bca91a6ea",
    "EndpointID": "072ab5f64199558831404539493a1130f2841e67a1bb0bc909df1b5e43036cbb",
    "Gateway": "172.20.0.1",
    "IPAddress": "172.20.0.3",
    "IPPrefixLen": 16,
    "IPv6Gateway": "",
    "GlobalIPv6Address": "",
    "GlobalIPv6PrefixLen": 0,
    "DNSNames": [
      "exercise-frontend-hardened",
      "frontend",
      "7f4b1ebaeaa9"
    ]
  }
}
```

Figure 45 - Hardened frontend network

```
(root@kali)-[/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# docker inspect exercise-backend-hardened | grep -A 50 "Networks"
"Networks": {
  "hardened_backend-network": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": [
      "exercise-backend-hardened",
      "backend"
    ],
    "MacAddress": "d6:e0:9f:48:07:65",
    "DriverOpts": null,
    "GwPriority": 0,
    "NetworkID": "2f9e291540a4df16d3a7e521095a7e8d181043dfd02b0fbdf1bb330dfe184e43",
    "EndpointID": "d52495694ac47d7b6cdfc9f8fec51c354053b235bc410a31e505b400ca4f1d8",
    "Gateway": "172.19.0.1",
    "IPAddress": "172.19.0.3",
    "IPPrefixLen": 16,
    "IPv6Gateway": "",
    "GlobalIPv6Address": "",
    "GlobalIPv6PrefixLen": 0,
    "DNSNames": [
      "exercise-backend-hardened",
      "backend",
      "e02a6bd420e5"
    ]
  },
  "hardened_frontend-network": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": [
      "exercise-backend-hardened",
      "backend"
    ],
    "MacAddress": "72:6a:0a:4b:42:8f",
    "DriverOpts": null,
    "GwPriority": 0,
    "NetworkID": "138b3f3ebd96c910f86ee83247659d71aa1995447a35f0016106e10bca91a6ea",
    "EndpointID": "3510e057151ae05818ae736f433551d330f6938a55da344ebc6b5ec73c6024f8",
    "Gateway": "172.20.0.1",
    "IPAddress": "172.20.0.2",
    "IPPrefixLen": 16,
    "IPv6Gateway": "",
    "GlobalIPv6Address": "",
    "GlobalIPv6PrefixLen": 0,
    "DNSNames": [
      "exercise-backend-hardened",
      "backend",
      "e02a6bd420e5"
    ]
  }
}
```

Figure 46 - Hardened backend network

### 3.4 TC-09 Resource Limits

TC09 identified that there were no CPU or memory limits configured for the containers, allowing them to consume unlimited host resources.

```
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# docker inspect exercise-backend | grep -E "Memory|NanoCPUs"
"Memory": 0,
"MemoryReservation": 0,
"MemorySwap": 0,
"MemorySwappiness": null,
```

Figure 47 - Original - no memory limit

```
(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# docker stats --no-stream
```

| CONTAINER ID | NAME              | CPU % | MEM USAGE / LIMIT   | MEM % | NET I/O        | BLOCK I/O       | PIDS |
|--------------|-------------------|-------|---------------------|-------|----------------|-----------------|------|
| 1e6f8c393770 | exercise-frontend | 0.01% | 8.527MiB / 1.926GiB | 0.43% | 1.41kB / 126B  | 64.5MB / 5.88MB | 7    |
| d4677c84e628 | exercise-backend  | 0.00% | 11.76MiB / 1.926GiB | 0.60% | 3.5kB / 1.75kB | 146MB / 75.2MB  | 14   |

Figure 48 - Resource usage

To address this issue, hardened docker-compose.yml applied resource limits to all three services using the mem\_limit and cpus. These limits were based on resource usage observed during testing, with additional capacity included to ensure stable operation.

```
root@kali: /home/kali/Downloads/cvs_pma_2026/cvs_pma_2026
Session Actions Edit View Help
GNU nano 8.7.1 docker-compose.yml

services:
  database:
    image: postgres:18.4
    container_name: exercise-database
    env_file: .env
    networks:
      - backend-network
    mem_limit: 256m
    cpus: 0.5
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "postgres"]
      interval: 50s
      timeout: 5s
      retries: 5
    volumes:
      - postgres-data:/var/lib/postgresql
      - ./backend/db-init.sql:/docker-entrypoint-initdb.d/01-db-init.sql:ro

  backend:
    image: exercise-backend:hardened
    container_name: exercise-backend-hardened
    env_file: .env
    ports:
      - "8081:8081"
    networks:
      - backend-network
      - frontend-network
    mem_limit: 512m
    cpus: 0.5
    depends_on:
      database:
        condition: service_healthy
    restart: unless-stopped

  frontend:
    image: exercise-frontend:hardened
    container_name: exercise-frontend-hardened
    ports:
      - "8080:80"
    networks:
      - frontend-network
    mem_limit: 128m
    cpus: 0.25
    depends_on:
      - backend
    restart: unless-stopped

networks:
  backend-network:
    driver: bridge
  frontend-network:
    driver: bridge

volumes:
  postgres-data:
```

Figure 49 - Hardened memory limits

This was verified using ‘docker inspect’, which showed the limits had been applied successfully. Further testing with ‘docker stats’ showed all containers operated comfortably within their allocated resources during normal use. This reduces any risk of resource exhaustion while maintaining application functionality.

```
(root@kali) - [ /home/kali/Downloads/cvs_pma_2026/cvs_pma_2026/hardened ]
# docker stats --no-stream
CONTAINER ID   NAME                                CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
b18fc37a30d7   exercise-frontend-hardened         0.01%     8.117MiB / 128MiB    6.34%     2.13kB / 4.79kB    41.9MB / 9.34MB    8
07fc677ab2f4   exercise-backend-hardened          0.00%     22.02MiB / 512MiB    4.30%     66.8kB / 66.8kB    48.8MB / 72.5MB    15
382982d8d76c   exercise-database                   0.02%     25.85MiB / 256MiB    10.10%    65.8kB / 62.7kB    76.6MB / 84.9MB    10
```

Figure 50 - Hardened resource usage

```

(root@kali)-[/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# docker inspect exercise-backend-hardened | grep -E "Memory|NanoCPUs"
"Memory": 536870912,
"MemoryReservation": 0,
"MemorySwap": 1073741824,
"MemorySwappiness": null,

(root@kali)-[/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# docker inspect exercise-frontend-hardened | grep -E "Memory|NanoCPUs"
"Memory": 134217728,
"MemoryReservation": 0,
"MemorySwap": 268435456,
"MemorySwappiness": null,

```

Figure 51 - hardened memroy limits

### 3.5 Final Testing

Lastly, a 'testing.sh' file was created to confirm the application remained fully functional and that the security requirements identified in Section1 had been successfully addressed. The script performs nine automated checks which verify that the hardening measures are implemented correctly.

```

(root@kali)-[/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# chmod +x testing.sh

(root@kali)-[/home/.../Downloads/cvs_pma_2026/cvs_pma_2026/hardened]
# ./testing.sh

HARDENED DEPLOYMENT VERIFICATION TESTING

[Test 1] Database connectivity -
PASS: Database reachable

[Test 2] Todos endpoint functioning -
PASS: Todos endpoint responding

[Test 3] Frontend serving correctly -
PASS: Frontend serving correctly

[Test 4] TC-03 - Database port NOT exposed to host -
PASS: Port 5432 not accessible from host

[Test 5] TC-05 - Containers running as non-root...
Backend /server process owner: appuser
Frontend container user: www-data
PASS: Both services running as non-root

[Test 6] TC-07 - Network isolation (frontend should not reach the database) -
PASS: Frontend isolated from backend-network

[Test 7] TC-09 - Resource limits applied -
PASS: Backend memory limit set

[Test 8] TC-10 - Health checks reporting healthy -
PASS: Containers report healthy

[Test 9] Security headers present (bonus finding) -
PASS: Security headers present

TESTING COMPLETE

```

Figure 52 - Testing

```

(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# hadolint hardened/backend/Dockerfile
hardened/backend/Dockerfile:3 DL3008 warning: Pin versions in apt get install. Instead of 'apt-get install <package>' use 'apt-get install <package>=<version>'
hardened/backend/Dockerfile:3 DL3015 info: Avoid additional packages by specifying '--no-install-recommends'

(root@kali)-[/home/kali/Downloads/cvs_pma_2026/cvs_pma_2026]
# hadolint hardened/frontend/Dockerfile
hardened/frontend/Dockerfile:3 DL3008 warning: Pin versions in apt get install. Instead of 'apt-get install <package>' use 'apt-get install <package>=<version>'

```

Figure 53 - Hardened hadolint

## Conclusion

The assessment identified ten security weaknesses in the application where appropriate hardening measures were implemented for each one of them. The findings were verified using automated tools and manual testing. Functional testing also confirmed that the application remained fully operational after the hardening measures were applied. Overall, the assessment shows the importance of combining secure image design, network configuration and runtime hardening to improve container security.

## Link to Video

[Meeting with NAIR, ADITI \(PGT\)-20260621\\_164651-Meeting Recording.mp4](#)

## References

CIS, Center for Internet Security (2025). *CIS Docker Benchmarks*, v1.8.0 [online] Available at: <https://www.cisecurity.org/benchmark/docker>.

Docker Documentation. (2025). *Best practices*. [online] Available at: <https://docs.docker.com/build/building/best-practices/#decouple-applications>.

Karunaratne, A. (2022). *PHP 7.4 releases, Docker images, source code downloads, and Windows binary downloads*. [online] PHP.Watch. Available at: <https://php.watch/versions/7.4/releases> [Accessed 20 Jun. 2026].

OWASP (2024). *HTTP Headers - OWASP Cheat Sheet Series*. [online] [cheatsheetseries.owasp.org](https://cheatsheetseries.owasp.org). Available at: <https://cheatsheetseries.owasp.org/cheatsheets/HTTP-Headers-Cheat-Sheet.html>.

PostgreSQL Documentation. (2026). *pg\_isready*. [online] Available at: <https://www.postgresql.org/docs/current/app-pg-isready.html> [Accessed 21 Jun. 2026].

Souppaya, M., Morello, J. and Scarfone, K. (2017). Application container security guide. *Application Container Security Guide*. [online] doi:<https://doi.org/10.6028/nist.sp.800-190>.

Appendices

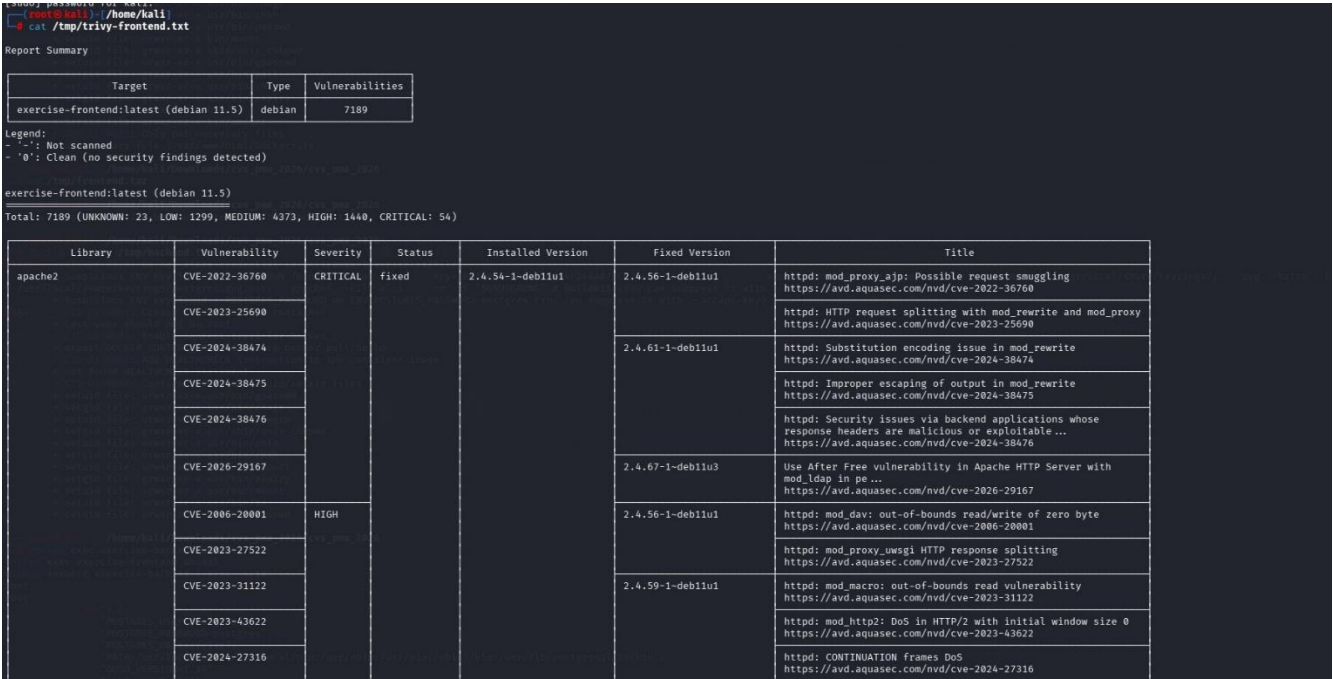


Figure 54 - Trivy